

ProMBA 논문의 상세 분석:
*Simplifying Mixed Boolean-Arithmetic
Obfuscation by Program Synthesis and
Term Rewriting*



April 13, 2025

Abstract

본 문서는 본 논문이 제시하는 MBA (Mixed Boolean-Arithmetic) 난독화 복원 기법 *ProMBA*의 핵심 논지, 이론적 배경, 방법론, 구현 세부사항, 그리고 실험 평가 결과를 정석적이고 극도로 자세하게 분석한 것이다. 특히, 프로그램 합성과 항 재작성, 선형 MBA 간소화 모듈을 융합하여 동적 규칙 학습 방식을 통해 다양한 난독화 패턴에 대응하는 접근법과 그 성능 및 한계점, 향후 연구 과제를 살펴본다.

1 개요 및 핵심 주장

본 논문은 간단한 프로그램 식을 복잡하고 난독화된 MBA 표현식으로 변환하는 일반적인 난독화 기법에 대응하기 위해, 프로그램 합성(Program Synthesis)과 항 재작성(Term Rewriting)을 결합한 새로운 MBA 복원 도구인 ProMBA를 제안한다.

- **핵심 아이디어:** 난독화된 MBA 표현식을 여러 하위 표현식으로 분할한 뒤, 각 하위 표현식에 대해 동적으로 합성을 수행하여 원래 의미와 동치인 더 간단한 식으로 치환하며, 이 과정에서 얻은 변환 규칙을 누적하여 전체 표현식을 점진적으로 단순화한다.
- **주요 장점:** 기존 도구들이 보이는 범용성 부족, 유연성 한계, 확장성 문제, 정확성 미보장의 한계를 극복하며, 다양한 비선형 MBA 표현식에 대해 높은 복원 성공률과 비교적 간결한 결과를 산출한다.

2 이론적 배경

2.1 Mixed Boolean-Arithmetic (MBA) 표현식 및 부울-산술 대수

- MBA 표현식은 부울 연산(예: $\wedge, \vee, \neg, \oplus$)과 산술 연산(예: $+, -, \times, /, \%$, $\ll, \gg$)을 포함하며, $(x \wedge y) - (x \vee y)$ 와 같은 MBA 다항식 MBA 비다항식 MBA 선형식, MBA 선형식 $+ y - 2(x \vee y) + 1$)으로 표현된다.
- 부울-산술 대수의 이론적 배경에 따르면, 모든 산술 및 논리 연산은 충분히 높은 차수의 다항식 형태로 표현될 수 있으므로, MBA 난독화 기법은 간단한 식을 다항식으로 치환해 코드의 의미를 보존하면서도 해석을 어렵게 만든다.

2.2 기존 MBA 복원 기법 및 한계점

- **항 재작성 기반 기법:** 수작업으로 작성된 규칙 집합에 의존하여 MBA 식을 단순화하지만, 새로운 난독화 방식에 대응하기 어렵고 범위가 제한적이다.
- **합성 기반 기법:** 입력-출력 예제를 통해 복원 식을 찾으나, 확률적 탐색에 의존하여 정확성 보장이 어렵다.
- **대수적 방법:** 선형 MBA 복원에 강점을 가지나, 비선형 또는 복합 연산이 포함된 경우 효율성이 급격히 떨어진다.

3 방법론: ProMBA 접근법

본 논문에서 제안한 ProMBA는 다음과 같은 단계로 동작한다.

3.1 전체 시스템 개요

1. **선형 MBA 간소화:** 난독화된 표현식 P 에서 선형 MBA 부분식을 찾아 기존의 선형 복원 도구 (SimbaD 등)를 통해 빠르게 단순화한다.
2. **부분식 선택 및 합성:** 난독화된 표현식의 비선형 하위 표현식 중, 제한된 높이 (h)를 갖는 부분식을 선택하고, 해당 부분식 r 에 대해 SyGuS (Syntax-Guided Synthesis) 문제로 동등하지만 더 간단한 식 r' 를 합성한다.
3. **동등 규칙 학습:** 합성을 통해 얻은 $r \rightarrow r'$ 변환을 일반화하여 규칙 집합 E 에 추가하고, SMT 솔버 (Z3)를 사용해 등가성을 철저히 검증한다.
4. **전역 재작성 및 반복:** 학습된 규칙을 전체 표현식에 적용하여 재작성 (rewrite)을 수행한 후, 선형 MBA 간소화를 다시 적용한다. 이 과정을 더 이상 단순화가 불가능할 때까지 반복한다.

3.2 온더플라이 동적 규칙 학습

- 기존 재작성 기법은 미리 고정된 규칙 세트를 사용하지만, ProMBA는 동적 규칙 학습 (*on-the-fly learning*)을 통해 표현식마다 필요한 변환 규칙을 즉석에서 찾아내어 누적함으로써, 새로운 난독화 패턴에도 유연하게 대응한다.
- 이 과정은 `ChooseSubExpr` 함수로 대상 부분식을 추출하고, `Synthesize` 절차를 통해 합성을 시도하며, 성공 시 `GeneralizeRule` 과정을 거쳐 일반화된 규칙을 얻는 방식으로 진행된다.

3.3 의사코드 예시

논문에서는 알고리즘 1과 2를 통해 전체 복원 과정을 제시한다. 의사코드의 개요는 다음과 같다:

```
p ← SimplifyLinearMBAs(P);
h ← 1; E ← ;
while (h ≤ Height(p)) do
  W ← Positions(p);
  while (W ≠ ∅) do
```

```

pos ← remove(W);
(r, ) ← ChooseSubExpr(p|pos, h);
r' ← Synthesize(r);
if (r' and |r'| < |r|) then
    E ← E {GeneralizeRule(r → r')};
    p ← p[pos ← (r')];
    p ← ApplyRule(E, p);
    p ← SimplifyLinearMBAs(p);
    W ← Positions(p);
end if;
end while;
h ← h + 1;
end while;
return p;

```

4 구현 세부사항

4.1 주요 구성 요소

- **SimbaD:** 선형 MBA 식을 대수적 방법(진리표 및 선형 결합 등)을 통해 간단히 복원하는 모듈로, 기존 연구에서 검증된 기술을 사용한다.
- **SimbaS:** SyGuS 기반의 프로그램 합성 엔진으로, 부울-산술 연산 및 비트벡터 문제에 특화되어 있어, 대상 부분식의 동등하지만 간단한 식을 열거형 탐색 방식으로 합성한다.
- **Z3 SMT Solver:** 합성된 변환 규칙과 최종 복원 결과의 등가성을 엄밀하게 검증하여, 복원 결과의 정확성을 보증한다.

4.2 구현 환경 및 언어

- 전체 시스템은 OCaml로 구현되었으며, 약 2800라인의 코드로 구성된다.
- 합성 문제는 SyGuS 포맷으로 인코딩되어 외부 합성 엔진(SimbaS 등)에 전달된다.
- 보조 도구로 Python 스크립트와 CIL (C Intermediate Language) 라이브러리를 활용하여 입력 표현식을 파싱하고 분석한다.

5 실험 평가

5.1 데이터셋 및 실험 환경

- **MBA-Solver 데이터셋:** 2011 개의 난독화 표현식 (선형 1008 개, 비선형 1003 개). 평균 변수 수 2.8 개, AST 노드 크기 평균 약 192.
- **QSynth 데이터셋:** Tigress 난독화기로 생성된 500 개의 비다항식 MBA 표현식. 평균 변수 수 2.3 개, 크기 평균 245 노드 (최대 2593 노드).
- **Loki 데이터셋:** Loki 난독화기로 생성된 1500 개의 표현식. 특히 난이도 높은 1000 개는 상수가 없는 다항식 MBA, 추가 500 개는 상수 및 모든 연산 포함. 평균 크기는 약 1568 노드 (최대 34553 노드).
- 실험은 Intel Xeon 2.6GHz, 256GB 메모리의 Linux 머신에서 진행됨.

5.2 비교 대상 및 성능 지표

- **비교 대상:** ProMBA vs MBASolver vs Syntia (신경망/합성 기반).
- **평가지표:**
 1. 복원된 표현식의 크기 (AST 노드 수),
 2. 성공률: 복원 결과가 원본 식과 동등하며 원본보다 간결하거나 같을 경우,
 3. 실행 시간: 각 표현식을 복원하는 데 걸린 시간.

5.3 실험 결과 요약

Table 1은 각 데이터셋에 대한 성능 비교 결과를 보여준다.

Table 1: ProMBA, MBASolver, Syntia의 성능 비교 (평균 값)

2*데이터셋	크기 (AST 노드)			성공률			실행 시간 (초)
	ProMBA	MBASolver	Syntia	ProMBA	MBASolver	Syntia	
MBA-Solver	11.76	21.67	4.61	80.3%	25.2%	17.5%	65.6
QSynth	17.48	77.71	4.72	62.8%	4.2%	22.8%	241.0
Loki	3.51	866.25	3.10	97.2%	0.13%	74.4%	100.0
전체	9.39	344.5	3.55	84.4%	13.2%	39.4%	100.4

주요 결과:

- ProMBA는 전체 복원 성공률 84.4%로 MBASolver(13.2%)와 Syntia(39.4%)를 월등히 능가함.
- 출력 식의 크기는 ProMBA가 평균 9.39 노드로 원본(약 9.64 노드)과 비슷하거나 더 간결한 반면, MBASolver는 평균 344.5 노드로 크게 부풀어 있음.
- 실행시간에서는 ProMBA가 데이터셋에 따라 다르지만, 전반적으로 MBASolver보다 효율적이며, 합성기에 따라 다르게 나타날 수 있음(합성 엔진으로 SimbaS 사용 시 평균 66초 내외).

6 한계점 및 향후 과제

- **합성 및 SMT 검증 확장성:** 복잡한 부분식에 대해 SyGuS 합성기와 SMT 솔버가 제한 시간 내에 해를 찾지 못하는 경우가 있어, 일부 사례에서 복원이 실패함.
- **매우 큰 표현식 처리:** Loki 데이터셋과 같이 수만 노드에 달하는 경우, 부분식 선택 및 반복 적용이 시간 면에서 부담이 될 수 있음.
- **Grammar 및 연산자 확장:** 현재 지원하는 연산자의 집합에 제한이 있을 수 있으므로, 새로운 난독화 기법에 대응하기 위해 grammar의 동적 확장이 필요함.
- 향후에는 합성기의 효율 개선, 병렬화, 그리고 SMT 솔버 성능 향상 등을 통해 전체 시스템의 확장성과 속도를 더욱 개선할 여지가 있다.

7 관련 연구 비교

- **항 재작성 기반 기법 (SSPAM 등):** 수작업 규칙의 한계로 새로운 난독화 패턴에 취약함.
- **신경망 기반 기법 (NeuReduce):** 입력-출력 샘플에 의존하여 정확성 보장이 어렵고, 새로운 사례에 대한 일반화가 미흡함.
- **대수적 방법 (Arybo, MBA-Blast, MBASolver, GAMBA):** 선형 및 일부 다항식 MBA에 효과적이거나, 비선형 복합 연산에 대해 자동 복원이 어려움.
- **합성 기반 기법 (Syntia, Xyntia, QSynth):** 확률적 탐색에 의존하여 일부 케이스에서 복원 성공하나, 샘플 제한으로 인한 정확성 문제와 확장성 이슈가 존재함.

ProMBA는 동적 규칙 학습과 프로그램 합성, 항 재작성의 결합을 통해 이러한 기존 접근법들의 한계를 보완하며, 특히 범용성, 정확성, 그리고 확장성 측면에서 큰 진보를 이루었다.

8 결론

본 논문은 MBA 난독화 복원을 위해 프로그램 합성과 항 재작성, 그리고 선형 MBA 간소화를 융합한 ProMBA 시스템을 제안하였다.

- ProMBA는 동적 규칙 학습(*on-the-fly rule learning*)을 통해 다양한 난독화 표현식에 대응하며, 복원 결과의 정확성(soundness)을 SMT 검증으로 보증한다.
- 실험 결과, ProMBA는 기존 도구들에 비해 높은 성공률과 간결한 출력식을 산출하며, 특히 복잡한 Loki 데이터셋에서 탁월한 성능을 보였다.
- 다만, 복잡한 부분식에 대한 합성과 SMT 검증의 확장성이 향후 개선되어야 할 과제가 확인되었다.

향후 과제: 향후 더 강력한 합성 기법과 SMT 검증 기술의 도입, 병렬 처리 및 grammar 확장 등으로 ProMBA의 성능과 확장성을 높이고, 전체 프로그램 단위로 확장 가능한 난독화 복원 체계를 구축할 필요가 있다.

참고문헌: 본 분석에서는 논문 본문 내 페이지번호 및 문단 번호(예: [L15-L23], [L289-L293])를 참조하여 내용을 작성하였으므로, 원문을 참고할 것을 권장한다.